

Version 2 — AI QA Testing Agent

What Is This?

An AI agent that **tests web applications like a human QA tester would** — but automatically. Give it a website URL, and it will:

- 1. Open the site in Chrome
- 2. Look at the page and understand what it sees (forms, buttons, dropdowns)
- 3. Fill in fields with test data
- 4. Click buttons, select options, upload files
- 5. Check if everything worked correctly
- 6. Report what it found — including any bugs

No scripts. No recordings. No hardcoded steps. The AI figures it out on its own.

Tech stack: Python + OpenAI Agents SDK + Chrome DevTools MCP (26 browser tools) + GPT-5

Why Version 2?

Version 1 proved the concept works — the AI can genuinely test web applications. But it was **too expensive to run regularly**. Every time the agent took an action, it had to re-read its entire conversation history, which kept growing. By action 46, it was re-reading the equivalent of a 50-page document every single step.

Version 2 solves this with **5 optimizations** that cut cost in half:

	Version 1	Version 2
Cost per page tested	\$0.25 - \$0.43	\$0.13
Time per page	3 - 4.5 minutes	2 minutes
Steps needed	27 - 46	19
Memory management	None (keeps growing)	Summarizes old work, keeps only recent context
Budget control	None	Real-time cost tracking, automatic stop at limit

How It Works

Think of it like hiring a QA tester with a notepad:

- 1. **The tester (AI)** opens the website and looks at the page
- 2. **The manager (our code)** watches over the tester's shoulder
- 3. After each action, the manager:
 - Updates the notepad summary ("Photo uploaded. First Name filled. 5 fields left.")
 - Throws away old detailed notes the tester doesn't need anymore
 - Checks the budget — stops if getting too expensive
- 4. The tester reads the notepad + sees the current page, then decides the next action

5. When all fields are done, the tester writes a final report with results and element locators (XPaths)

The AI makes all the decisions (what to click, what to type, how to handle errors). Our code just manages the paperwork between decisions.

Technical: Architecture



v1 vs v2 — the core difference:

v1 (black box):	v2 (managed loop):
SDK handles all 120 turns internally – we have zero control over what happens between actions.	Our code runs one turn at a time:
We just wait and get the final result.	1. Run single turn
turn	2. Compact history (remove old data)
Cost: unpredictable	3. Update rolling summary
	4. Check budget
	5. If done → stop, else → next
	Cost: controlled, ~50% cheaper

The 5 Optimizations

1. Smarter Instructions

Taught the AI shortcuts: check for hidden file inputs before clicking upload buttons, skip fields that already have correct values, use JavaScript fallbacks immediately for tricky inputs. **Saves: ~16 unnecessary steps per page**

Technical detail: These are prompt-level changes in `prompts.py` (Lever 1). v1's prompt had the agent discover workarounds through trial and error (e.g., 9 turns to figure out file upload uses a hidden input). v2's prompt teaches these patterns upfront with worked examples so the agent goes straight to the right approach.

2. Less Verbose Thinking

The AI used to write a full paragraph of reasoning before every action — even simple ones like typing a name. Now it's concise for easy actions, detailed only when something goes wrong. **Saves: ~30% on AI output costs**

Technical detail: Output tokens cost \$10/MTok — the most expensive token type. v1 required full THOUGHT/ACTION/OBSERVATION format on every turn. v2 allows short-form reasoning for simple actions (Lever 2). Also reduced worked examples from 5 to 3, saving ~400 tokens per turn in the system prompt.

3. Cleaner Page Reading

When Chrome reports what's on the page, it includes everything — logos, decorative dividers, 190 countries in a dropdown list. We filter this down to only what matters: form fields, buttons, headings. **Saves: up to 99% on large dropdown snapshots**

Technical detail: `snapshot_filter.py` (Lever 3) processes the accessibility tree before the AI sees it. It removes decorative elements (images, separators), structural wrappers (empty divs), and collapses long option lists (190 countries → 5 shown + "185 more"). The country dropdown snapshot went from 19,761 tokens to ~200 tokens. The prompt also instructs the agent to close dropdowns before taking snapshots to prevent bloat.

Filter rules:

Rule	What Gets Removed	Tokens Saved
Decorative roles	Images, separators, presentation elements	~50-200/snapshot
Structural roles	Empty divs/spans (wrapper noise)	~100-300/snapshot
Option collapsing	190 options → "5 shown + 185 more"	~18,000 for country dropdown
Empty lines	Blank lines between elements	~20-50/snapshot

4. Memory Management

Instead of keeping the full history of every action (which grows to 50+ pages), we keep a short summary + only the last 2 actions in detail. The AI has everything it needs without the bloat. **Saves: ~40% on total input costs**

Technical detail: `compactor.py` + `orchestrator.py` (Lever 4). After every turn, `compact_history()` runs:

- 1. Keeps the first item (original task message)
- 2. Replaces all old turns with a rolling summary: `"CONTEXT: Photo uploaded. FirstName=ROMAN. 5/7 fields done."`
- 3. Keeps the last 2 turns raw (so the AI has immediate context)
- 4. Runs `snapshot_filter` on any kept tool results

Context stays flat at ~9-12K tokens instead of growing to 53K.

WITHOUT compaction (v1):		WITH compaction (v2):	
Turn 1:	6K tokens	Turn 1:	6K tokens
Turn 10:	17K tokens	Turn 10:	~10K tokens
Turn 20:	28K tokens	Turn 20:	~11K tokens
Turn 46:	53K tokens	Turn 46:	~12K tokens (flat)

Trade-off: Compaction changes the input prefix every turn, which reduces OpenAI's cache hit rate (v1: 93% → v2: 50-66%). But fewer total tokens more than compensates — net result is still ~50% cheaper.

GPT-5 reasoning items: GPT-5 produces `reasoning` type items paired with `message` items. The compactor backs up the cut point to include preceding reasoning items, or the API returns a 400 error.

5. Cheaper Model for Simple Tasks (Planned)

Simple actions (typing a name, clicking a button) don't need the most powerful AI model. We plan to route these to a cheaper model while keeping the expensive model for complex diagnosis. **Estimated savings: 50-70% on simple actions**

Technical detail: Lever 5 (not yet built). Would use `gpt-5-mini` (\$0.25/\$2 per MTok) for simple fills/clicks and `gpt-5` (\$1.25/\$10 per MTok) for failures, dropdowns, and diagnosis. Requires Lever 4 (orchestrated loop) as foundation since we need turn-level control to switch models.

What Can It Do?

Test Modes

Mode	What It Does	Cost	Use Case
Safe Test	Opens the page, looks around, describes what it sees. No interaction.	~\$0.02	Verify the agent can reach the site
Recon	Explores the page and extracts element locators (XPaths). No filling.	~\$0.04-0.08	Get XPaths for a new page without interacting

Mode	What It Does	Cost	Use Case
POC Short	Fills forms and extracts XPaths. Limited to 30 actions.	~\$0.10-0.20	Quick test on external sites
POC	Full page test — fills everything, diagnoses issues, extracts XPaths.	~\$0.13-0.25	Thorough single-page testing
Full	Tests all 6 pages of the application (not yet tested).	~\$1.5-3.0 est	Complete regression test

Running a Test

```
python run.py                                # Safe test
on TECU (default)
python run.py poc                            # POC test
on TECU
python run.py recon "https://example.com" "My App" # Recon on
any site
python run.py poc_short "https://example.com" "My App" # Quick
test on any site
```

Any website URL can be tested — no configuration needed. If no URL is provided, defaults to the TECU Credit Union app (configurable in `config.py`).

Technical detail: `run.py` reads mode from `sys.argv[1]`, URL from `sys.argv[2]`, app name from `sys.argv[3]`. Defaults come from `config.py` (`TARGET_URL`, `APP_NAME`). Turn limits per mode are defined in `TURN_LIMITS` dict. Mode determines the task message sent to the AI:

- `safe_test`: "Navigate and describe what you see"
- `recon`: "Explore, discover elements, extract XPaths. Do NOT click or fill."
- All others: "Test the web application. Do not click 'Save & Continue'."

An XPath extraction nudge is injected `XPATH_NUDGE_BEFORE_END` turns before the limit (e.g., turn 27 of 30) to ensure XPaths are captured even if the agent hasn't finished filling all fields.



Test Results

Primary Target: TECU Credit Union Loan Application

Version	Steps	Cost	Time	Result
v1 (best)	27	\$0.25	3 min	All 7 fields filled
v1 (worst)	46	\$0.43	4.5 min	Same result, wasted steps
v2 (best)	19	\$0.13	2 min	All 7 fields + XPaths extracted

The agent found real bugs in the application:

- Last Name input produces garbled text when typing (buggy event handler)
- Upload button doesn't trigger file chooser (hidden input needed)
- Masked mobile input doesn't accept standard fill commands

External Sites Tested

Website	Mode	Turns	Cost	Result
DemoQA Practice Form	recon	4	~\$0.03	XPaths extracted successfully
Eventbrite Signup	safe_test	3	~\$0.02	Worked, saw cookie banner
ParaBank Banking App	poc	16	\$0.15	Filled 10 registration fields + XPaths
Karnataka Seva Sindhu (Govt)	poc_short	30	~\$0.10	Explored portal, extracted navigation XPaths
BookMyShow	poc_short	—	—	Blocked by bot detection (Cloudflare)

The agent works on **any website** — not just the one it was built for.

Technical: Full Run History								
Run	Date	Version	Mode	Turns	Cost	Cache Hit	Duration	Notes
1	Mar 6	v1	poc	27	\$0.27	85%	180s	Page 1, clean
2	Mar 6	v1	poc	30	\$0.25	88%	190s	Page 1
3	Mar 7	v1	poc	46	\$0.43	93%	270s	Page 1, too many turns
4	Mar 9	v2	safe_test	3	\$0.02	58%	18s	First v2 test
5	Mar 9	v2	poc	50	\$0.23	73%	300s	Loop: 22 turns on Branch dropdown
6	Mar 11	v2	poc	19	\$0.13	66%	121s	Best run — all fields + XPaths
7	Mar 11	v2	recon	4	~\$0.03	99%	~30s	DemoQA
8	Mar 16	v2	poc	16	\$0.15	51%	~120s	ParaBank — filled 10 fields

What It Produces

Every test run generates:

- 1. **Results Table** — Each field tested, the value entered, whether it worked, and any notes
- 2. **XPaths** — Real element locators extracted from the live page (not guessed)
- 3. **Issues List** — Bugs, API errors, accessibility problems discovered
- 4. **Cost Summary** — Exact cost breakdown for the run

Example output:

Field	Value	Status	Notes
First Name	ROMAN	filled	Shows transformed uppercase
Email	ROMAN@EXAMPLE.COM	filled	Shows transformed uppercase
Mobile Number	620-1234	filled	Auto-formatted with hyphen
Branch	200 - TECU - COUVA	filled	Selection reflected on button
Save & Continue	-	skipped	Per instruction: do not click

XPaths:
firstName: //input[@name='firstName' and @placeholder='Enter first name' and @id='firstName']
email: //input[@name='email' and @placeholder='Enter email address' and @id='email']

Technical: Output Files

Every run saves two files in `version_2/runs/`:

- 1. `output_v2_{mode}_{timestamp}.txt` — Human-readable report with run metadata, turn log table, and the agent's final output (RESULTS, XPATHS, ISSUES sections).
- 2. `turns_v2_{mode}_{timestamp}.json` — Machine-readable turn log with per-turn token counts, actions, cache ratios, and rolling summary. Used by `dashboard.py` for analytics.

Usage data also logged to Excel via `state/usage_tracker.py` for cumulative cost tracking across all runs.

Project Files

version_2/
run.py
config.py
prompts.py

– Start a test (command-line entry point)

– All settings (model, budget limits, pricing)

– The AI's instructions and behavior rules

- orchestrator.py
- compactor.py
- snapshot_filter.py
- runs/
- The main engine (runs tests step by step)
- Memory management (keeps context small)
- Page cleanup (removes visual noise)
- Test output files (results, logs)

Technical: File Details

File	Lines	What It Does	Optimization
run.py	~27	Parses CLI args (mode, URL, app name), calls <code>run_orchestrated()</code> . Falls back to <code>TARGET_URL</code> and <code>APP_NAME</code> from config.	—
config.py	~58	All constants: <code>MODEL</code> , <code>TURN_LIMITS</code> (per mode), <code>MAX_BUDGET</code> , <code>COMPACT_AFTER_TURNS</code> , <code>KEEP_LAST_N_TURNS</code> , <code>PRICING</code> (GPT-5 rates), <code>MAX_DROPDOWN_OPTIONS</code> , <code>RUNS_DIR</code> . Nothing is hardcoded in other files.	—
prompts.py	~234	System prompt split into sections: IDENTITY, REASONING FORMAT (concise vs verbose), THREE MENTAL STAGES (Explore/Realize/Act/Diagnose), WORKED EXAMPLES (3 optimized patterns), INTERACTING WITH ELEMENTS (skip defaults, multi-form handling), FAILURE FAST-PATHS, XPATH EXTRACTION (<code>evaluate_script</code> template), GUARDRAILS (max retries, loop detection).	Lever 1, 2, 3
orchestrator.py	~396	The heart. Launches MCP server, creates Agent, runs turn-by-turn loop via <code>Runner.run(max_turns=1)</code> with error handler for <code>MaxTurnsExceeded</code> . Between turns: compacts history, updates rolling summary, checks budget (warning at 50%, hard stop at limit), injects XPath nudge near end. <code>LiveTurnLogger</code> (RunHooks) prints per-turn stats table and tracks cumulative tokens for cost calculation. Saves output files + Excel log at end.	Lever 4

File	Lines	What It Does	Optimization
compactor.py	~216	<code>compact_history()</code> : keeps first item (task) + rolling summary + last N turns. <code>update_summary()</code> : extracts tool name/target from each turn and appends short description ("Filled 'Roman'"). <code>_get_last_n_turns()</code> : finds turn boundaries (assistant messages), backs up to include GPT-5 reasoning items. <code>_filter_snapshots_in_items()</code> : runs <code>snapshot_filter</code> on kept tool results.	Lever 4
snapshot_filter.py	~169	<code>filter_snapshot()</code> : removes DECORATIVE_ROLES (image, separator, presentation), STRUCTURAL_ROLES (generic, group — only if no useful text), collapses option lists beyond <code>MAX_DROPDOWN_OPTIONS</code> with count summary. Heuristic <code>_looks_like_snapshot()</code> detects accessibility tree output.	Lever 3
Key technical decisions: <code>MaxTurnsExceeded</code> handling: SDK raises exception when <code>max_turns=1</code> is hit. Solved with <code>error_handlers={"max_turns": _on_max_turns}</code> which returns <code>RunErrorHandlerResult(final_output="__TURN_LIMIT__")</code> so <code>Runner.run</code> returns normally with all history intact. - GPT-5 reasoning items: Messages have paired <code>reasoning</code> type items. Compactor's <code>_get_last_n_turns()</code> backs up the cut point to include any preceding reasoning item, preventing API 400 errors. - Rolling summary is append-only and grows ~1-5 tokens per turn (vs raw history growing 500-20,000 per turn).			

Current Limitations

1. Repetition on tricky elements

The AI sometimes retries the same action on a difficult element (like a custom dropdown) because the memory summary doesn't capture failure details. In one run, it tried the same dropdown 22 times, wasting ~\$0.15.

Technical: Root cause and planned fixes

With `KEEP_LAST_N_TURNS=2`, the agent forgets previous failures after compaction. The rolling summary says "Clicked 1_33." but not "Clicked 1_33 — FAILED 3 times."

Fix A — Loop Detection (circuit breaker in orchestrator.py): Track last 5-6 actions. If same element appears 3+ times, inject: "LOOP DETECTED: You tried element 1_33 three times. Skip it, move on." Active kill switch — stops loops after 3 attempts instead of 22.

Fix B — Enriched Summary (in compactor.py): Change summary from "Clicked 1_33." to "Branch dropdown (1_33): tried click, press_key, evaluate_script — FAILED. SKIP." Preventive — agent reads summary and knows not to retry.

Fix C — SQLite Cold Storage (new file): Save failure data across runs. Day 1: agent wastes turns on Branch → saved to DB. Day 2: orchestrator checks DB before run → injects hint. Zero wasted turns.

2. Multi-form pages

Pages with both a login form and a registration form can confuse the agent. It may try to fill the login section first. Partially fixed with smarter instructions.

Technical: Prompt now includes rules: "If the page has multiple forms, prioritize the one matching the URL path. IGNORE login forms unless explicitly instructed. Still extract XPaths for ALL forms." Tested on ParaBank — agent filled login first (3 turns wasted), then opened new tab for registration. Needs further testing after prompt update.

3. Bot-protected sites

Sites with Cloudflare or aggressive bot detection may block the automated Chrome browser. Works on most sites but not all.

Technical: Three-tier bypass strategy

Tier 1 (flags): `--ignoreDefaultChromeArg=--enable-automation + --chromeArg=--disable-blink-features=AutomationControlled` — removes `navigator.webdriver=true` signal. Partially works (inconsistent on BookMyShow). Currently **disabled** — caused about:blank issues on some runs.

Tier 2 (real Chrome): Launch Chrome manually with `--remote-debugging-port=9222`, then connect MCP via `--browserUrl=http://127.0.0.1:9222`. No automation flags at all. Not yet implemented.

Tier 3 (stealth MCP): Use a stealth-focused MCP server instead of Google's. Last resort. Not explored.

4. Full 6-page run not yet validated

Individual page testing works well. Multi-page end-to-end testing is the next milestone.

5. Orphan Chrome processes

After Ctrl+C, Chrome processes from the MCP server may not be killed properly. Always clean up before running again:

```
ps aux | grep "chrome-devtools-mcp" | grep -v grep | awk '{print $2}' |
xargs kill 2>/dev/null
```

Cost Model

Approach	Cost per Full Run	Speed	Maintenance
Manual QA tester	~\$15-25/hour	30-60 min	High (human time)
Current Playwright scripts	\$0 (free)	30-60 sec	Medium (scripts break when UI changes)
AI Agent v2	~\$1.5-3.0 est	~20-25 min est	Low (adapts to UI changes)

The AI agent isn't replacing the free Playwright scripts for daily regression. It's for **intelligent testing** — finding issues that scripted tests miss, testing new pages without writing scripts, and adapting when the UI changes.

Technical: GPT-5 Token Pricing

Token Type	Rate (\$/MTok)	What It Is
Uncached input	\$1.25	New content the model hasn't seen before
Cached input	\$0.125 (90% discount)	Repeated prefix (system prompt, tool schemas)
Output	\$10.00	Model's response (reasoning, tool calls)

Per-turn input composition:

system_prompt

~1,400 tokens

– fixed, cached after turn 2

tool_schemas

~5,200 tokens

– 26 MCP tool definitions, always cached

task_message

~30 tokens

– fixed

rolling_summary

~50–200 tokens

– grows slowly

last 2 turns

~2,000–4,000

– changes every turn (partially cached)

Budget enforcement: `MAX_BUDGET` in config.py (currently \$0.50). Warning printed at 50% of budget. Hard stop when exceeded. `LiveTurnLogger.get_current_cost()` calculates real cost per turn using cached vs uncached split.

Dependencies

Package	Purpose
11 / 12	

Package	Purpose
Python 3.10+	Runtime
OpenAI Agents SDK (openai-agents)	Agent framework — manages LLM calls, tool routing
Chrome DevTools MCP	26 browser tools (click, fill, snapshot, network, console)
Node.js v20.19+	Runs the MCP server process
Google Chrome (stable)	The browser the agent controls
python-dotenv	Loads <code>OPENAI_API_KEY</code> from <code>.env</code>
openpyxl	Excel logging for usage tracking
structlog	Structured logging

What's Next

1. **Fix the repetition problem** — Detect when the agent is stuck and force it to move on (Fix A: loop detection circuit breaker)
2. **Better memory for failures** — Include "what went wrong" in the summary so the agent learns mid-test (Fix B: enriched rolling summary)
3. **Re-run TECU POC** — Validate cost improvement after fixes
4. **Full 6-page test run** — Test the complete TECU loan application end-to-end
5. **Cross-run learning** — Save lessons from each test so future runs start smarter (Fix C: SQLite cold storage)
6. **Cheaper model routing** — Use a budget model for simple actions, premium model for complex diagnosis (Lever 5)